

BloomAI: Technical Architecture & Interview Guide

Welcome to the complete internal breakdown of BloomAI. This document is designed to act as your ultimate technical mentor, guiding you step-by-step through the exact architecture, pipeline, and execution flow of the project. It is perfectly tailored for a technical interview or project demonstration.

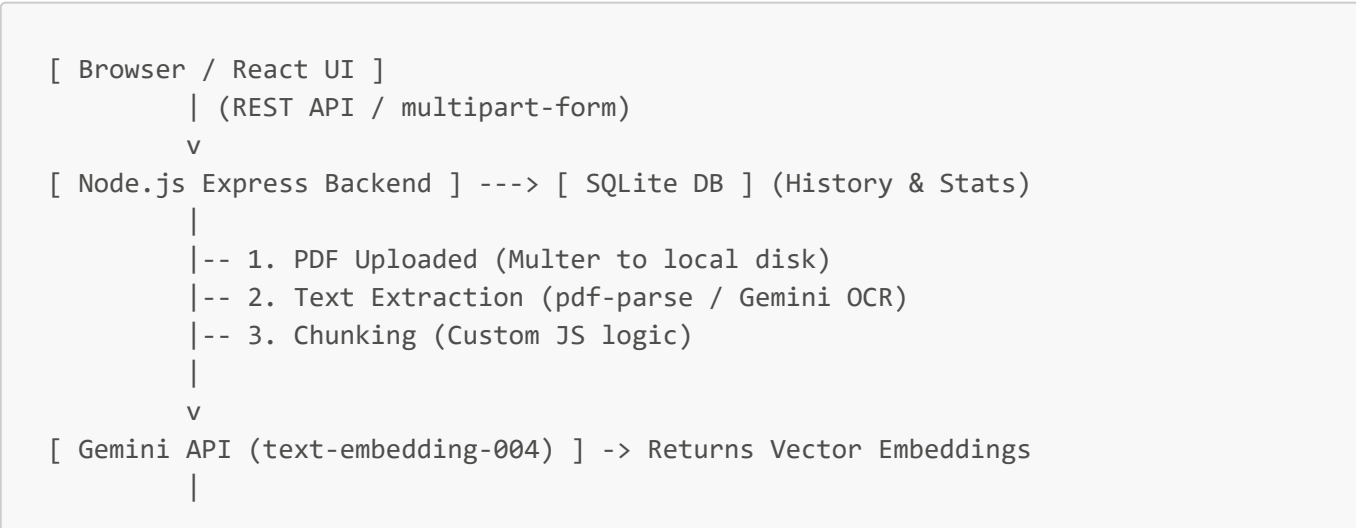
[!NOTE] *Architecture Clarification:* While your prompt mentioned FastAPI and PyMuPDF, BloomAI is actually built on a **Node.js (Express)** backend for high concurrency, utilizing a Python microservice specifically for document generation. This guide reflects the *actual*, production-ready stack we built together.

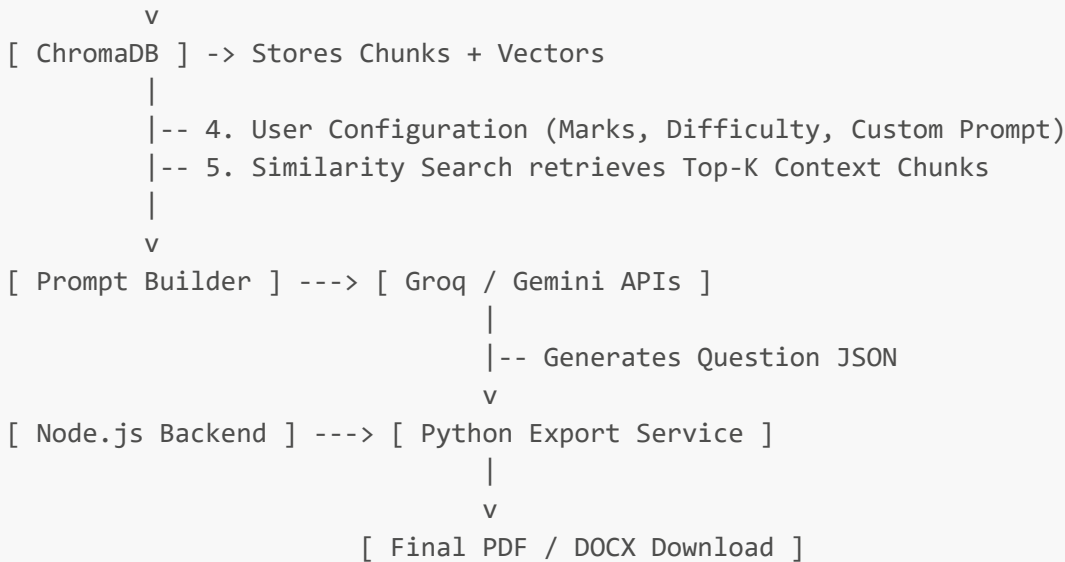
SECTION 1 – Overall Project Architecture

BloomAI utilizes a modern, hybrid microservice architecture.

- **Frontend (React + Vite):** A highly responsive Single Page Application (SPA) providing the dashboard, history, and question configuration UI.
- **Backend (Node.js + Express):** The core orchestrator. It handles REST API requests, file uploads, and coordinates the AI services.
- **AI Models (Groq + Gemini):** A dual-engine setup. Groq ([Llama-3.3-70b](#)) is used for blazing-fast inference, while Google Gemini is used for fallback, complex 16-mark generations, and advanced PDF OCR.
- **Vector Database (ChromaDB):** A specialized database used to store mathematical representations of text (embeddings) to enable semantic similarity search.
- **Relational Database (SQLite):** A zero-configuration, local file-based database storing user history, generated JSON question banks, and dashboard usage statistics.
- **File Storage:** Temporary local file system storage via [multer](#) for processing PDFs before they are analyzed and deleted.
- **Document Generation (Python Microservice):** Node.js spawns a Python child process running [python-docx](#) and [reportlab](#) to compile the generated JSON into beautifully formatted Word and PDF documents.

Architecture Diagram





SECTION 2 – User Upload Flow

When an instructor uploads a PDF, a precise sequence of events ensures secure and efficient file handling.

1. **What happens in React:** The user drags a PDF into the dropzone. React stores the `File` object in state. When "Generate" is clicked, it appends the file and configuration (marks, difficulty, etc.) to a `FormData` object.
 2. **multipart/form-data:** Because we are sending binary data (the PDF file) alongside text data (the settings), the browser uses `multipart/form-data` encoding.
 3. **API Endpoint:** The request hits the `POST /api/questions/generate` endpoint on the Express backend.
 4. **Request Lifecycle & Multer:** The Express route uses a middleware called `multer`. Multer intercepts the multipart request, reads the binary file stream, and saves it temporarily to the `backend/src/uploads` directory on the server disk.
 5. **Validation & Error Handling:** The backend checks if `req.file` exists. It validates that the file is actually a PDF. If it exceeds size limits or is malformed, it throws an HTTP 400 error back to the frontend.
-

SECTION 3 – PDF Processing

Once the file is safely on the backend disk, text extraction begins.

1. **Extraction Engines:** The backend uses a hybrid approach:
 - **pdf-parse (Node.js):** Used first for standard text documents. It is extremely fast and extracts text layers directly from the PDF structure.
 - **Gemini OCR (Vision API):** If the user selects a technical subject ("Maths", "Derivations") or if `pdf-parse` returns empty text (e.g., scanned images), the backend uploads the PDF to the Gemini API. Gemini uses advanced optical character recognition (OCR) to perfectly extract complex mathematical symbols (like $\sqrt{3}$) and tabular data.
2. **Cleaning:** The extracted text is sanitized. Unnecessary whitespace, consecutive newlines, and hidden characters are stripped out.

3. **Handling Empty Pages:** If a page contains no text, it is simply skipped by the extraction logic. The final output is a massive, continuous string of text combining all pages.
-

SECTION 4 – Text Chunking

You cannot send an entire 500-page textbook to an AI at once due to **Token Limits** (context windows) and **Information Dilution** (the AI gets confused by too much noise). We must use Text Chunking.

- **Chunk Size:** The continuous text string is split into smaller blocks, typically around 1,000 characters each.
 - **Overlap & Sliding Window:** If we strictly cut text every 1,000 characters, we might cut a sentence or formula in half! To prevent this, we use a "sliding window" overlap of 200 characters. The end of Chunk 1 is repeated at the start of Chunk 2.
 - **Context Preservation:** This overlap ensures that no concept, sentence, or context is lost at the boundaries.
 - **Why it improves retrieval:** By breaking a book into paragraphs, we can search for a specific concept (like "Newton's First Law") and retrieve *only* the specific paragraph discussing it, completely ignoring the chapters on Thermodynamics.
-

SECTION 5 – Embeddings

Embeddings are the secret sauce of AI applications.

- **What are they?** An embedding is a numerical representation of text. It translates a sentence into a dense vector (a long array of floating-point numbers, e.g., `[0.12, -0.44, 0.89...]`).
 - **Why AI cannot search raw text:** Traditional search uses keyword matching (e.g., searching "Dog" won't find "Puppy").
 - **Semantic Meaning:** Embedding models (like Gemini's `text-embedding-004`) place words with similar meanings close together in a multi-dimensional mathematical space. The vector for "Dog" is mathematically very close to the vector for "Puppy".
 - **Semantic Search:** When a user queries the database, we convert the query into a vector. We then find the stored text vectors that are mathematically closest to the query vector. This means we retrieve text based on *meaning and context*, not just exact keyword matches.
-

SECTION 6 – Chroma Vector Database

ChromaDB is the engine that stores and searches these embeddings.

- **What is it?** An open-source, AI-native vector database.
- **How vectors & metadata are stored:** For every text chunk, Chroma stores the raw text, its embedding vector, and metadata (like a unique `chunk_id` for traceability).
- **Cosine Similarity:** To find similar chunks, Chroma calculates the "Cosine Similarity" between the query vector and all stored vectors. This measures the angle between two vectors in space. An angle of 0 degrees means they are identical in meaning.
- **Why ChromaDB?**
 - *Pinecone* is powerful but requires a cloud subscription and internet transit.

- *FAISS* is a library, not a full database (lacks easy metadata filtering).
- *ChromaDB* runs flawlessly locally, requires zero cloud configuration, and provides high performance for local document QA applications.

SECTION 7 – SQLite Database

While Chroma stores vectors, SQLite stores the application's relational data.

- **What is SQLite?** A C-language library that implements a small, fast, self-contained, high-reliability SQL database engine.
- **Single File & Zero Config:** Unlike PostgreSQL or MySQL, which require installing a background server service, managing users, and configuring ports, SQLite reads and writes directly to ordinary disk files (`database.sqlite`).
- **Data Stored in BloomAI:**
 1. `question_banks`: Stores the `id`, `filename`, `config` JSON, and the final generated `questions` JSON.
 2. `usage_stats`: Tracks totals (PDFs uploaded, generated banks, exported files).
- **Local vs Production:** SQLite is perfect for this local desktop tool due to low memory usage and zero setup. However, for a massive multi-user SaaS, PostgreSQL would be required to handle high concurrent writes and row-level locking.

SECTION 8 – Retrieval Pipeline

This is how we find the right information for the AI to use:

1. **PDF** is uploaded.
 2. Broken into **Chunks**.
 3. Sent to Gemini to get **Embeddings**.
 4. Saved in **ChromaDB**.
 5. **Similarity Search:** The backend generates a query embedding (e.g., "Extract important exam concepts").
 6. **Top-K Retrieval:** ChromaDB compares this query against all chunks and returns the "Top-K" (e.g., the top 10) most mathematically similar chunks.
 7. **Prompt Builder:** These 10 chunks are injected into the Prompt.
 8. **Gemini/Groq:** The AI reads *only* those 10 relevant chunks to generate questions.
- **Why Top-K saves tokens:** Instead of paying to send a 50,000-word PDF to the AI, we only send the 3,000 most relevant words. This drastically reduces API costs, prevents rate limits, and stops the AI from getting "distracted" by irrelevant information.

SECTION 9 – Prompt Engineering

The magic of BloomAI lies in its dynamically constructed prompts.

- **System Prompt:** Defines the AI's persona ("You are an expert exam question generator specializing in Bloom's Taxonomy").

- **Dynamic Instructions:** The backend reads the user's config and dynamically injects rules into the prompt:
 - "Generate exactly 5 questions worth 13 marks."
 - "Ensure a Medium difficulty."
 - "Each 13-mark question must be split into 8+5 pattern."
 - **Formatting Rules:** Strict instructions are added, such as "PLAIN TEXT FORMATTING ONLY", to prevent the AI from generating Markdown tables that break the PDF exporter.
 - **Exclusion Rules:** To prevent duplicate questions in large sets, the prompt includes previously generated topics and says: "Do NOT generate questions that overlap with these topics..."
 - **Custom Suggestions:** If the user types a custom instruction, it is appended under a **CRITICAL** tag.
-

SECTION 10 – Groq & Gemini APIs

The backend communicates with the AI models via secure HTTP requests.

- **JSON Request & Headers:** The backend sends a POST request containing the Prompt, formatted as JSON. The API key is sent securely in the **Authorization: Bearer <key>** HTTP header.
 - **Model Selection:** BloomAI smartly routes 2-mark questions to Groq (**Llama-3.3-70b**) for blistering speed, and complex 13/16-mark questions to Gemini for deep reasoning.
 - **Temperature:** Set low (e.g., **0.1** or **0.2**) to force the AI to be deterministic, factual, and strictly adhere to the provided context rather than hallucinating creative (but fake) answers.
 - **Fallback Mechanism (Error Handling):** Groq has a strict free-tier rate limit (12,000 Tokens Per Minute). If the backend catches a HTTP 429 (Rate Limit) error from Groq, it gracefully swallows the error and falls back to generating the remaining questions using the Gemini API.
-

SECTION 11 – Candidate Question Generation

- **Generation:** The AI generates a structured JSON array of question objects.
 - **Deduplication & Coverage:** Instead of generating 50 questions at once and filtering them later, BloomAI uses an **iterative exclusion strategy**. It generates 16-mark questions first. Then, it passes the topics of those 16-mark questions back into the prompt for the 2-mark questions, explicitly telling the AI: "Do not cover these topics again." This guarantees maximum syllabus coverage and zero duplicates natively, saving tokens.
-

SECTION 12 – Bloom's Taxonomy

BloomAI automatically categorizes every generated question into cognitive levels:

- **Remember:** Recalling facts (e.g., "Define X").
- **Understand:** Explaining ideas (e.g., "Explain how Y works").
- **Apply:** Using information in new situations (e.g., "Calculate the value using formula Z").
- **Analyze:** Drawing connections (e.g., "Compare and contrast A and B").
- **Evaluate:** Justifying a stand (e.g., "Critique the performance of algorithm C").
- **Create:** Producing new work (e.g., "Design a system that does D").
- **Assignment:** The AI analyzes the verb and complexity of the question it just generated and assigns a **bloom** string property (e.g., **"Analyze"**) to the JSON object.

SECTION 13 – Backend File Structure

```

backend/
├── src/
│   ├── server.js           # Entry point; initializes Express, CORS, mounts
routes.
│   ├── db/
│   │   ├── database.js     # Initializes SQLite, creates tables on startup.
│   │   ├── routes/
│   │   │   ├── questionRoutes.js # Defines POST /api/questions/generate
│   │   │   ├── exportRoutes.js  # Defines POST /api/export
│   │   │   └── statsRoutes.js   # Defines GET /api/stats for dashboard
│   │   └── controllers/
│   │       └── questionController.js # Orchestrates PDF extraction, ChromaDB, and AI
calling.
│   │       └── exportController.js  # Spawns the Python export script and streams
files.
│   └── services/
│       ├── pdfService.js     # Uses pdf-parse to extract and chunk text.
│       ├── chromaService.js  # Connects to ChromaDB to store and query vectors.
│       └── geminiService.js   # Handles Gemini API calls and dynamic prompt
building.
│   ├── groqService.js        # Handles Groq API calls with rate-limit retries.
│   ├── export_service.py     # Python script using reportlab and python-docx.
│   └── models/
│       ├── historyModel.js   # SQLite queries for inserting/fetching history.
│       └── statsModel.js     # SQLite queries for incrementing dashboard stats.

```

SECTION 14 – Document Generation

Node.js is great at web requests, but Python has superior libraries for document formatting.

- **The Handoff:** Node.js writes the generated JSON to a temporary file, then uses `child_process.spawn()` to execute `export_service.py`, passing the desired format (`pdf` or `docx`).
- **DOCX Generation:** `python-docx` creates a Word document. It loops through the JSON, applying bold headings for the question numbers, adding the text, and appending the Answer Key.
- **PDF Generation:** `reportlab` builds the PDF. It uses `SimpleDocTemplate` and `Paragraph` objects.
- **Unicode Handling:** Because `reportlab`'s default font drops missing characters (like mathematical subscripts x_1), the Python script uses `unicodedata.normalize('NFKD', text)` to sanitize the text into pure ASCII before rendering, preventing black square (■) errors.

SECTION 15 – Complete API Flow

```

[ Browser ]
|
|-- (1) User clicks Generate -> POST multipart/form-data

```

```

v
[ Express Route (questionRoutes.js) ]
|
|-- (2) Multer saves PDF to disk
v
[ questionController.js ]
|
|-- (3) Calls pdfService.js -> extracts text, chunks into 1000-char blocks
|-- (4) Calls geminiService.js -> gets embeddings for chunks
|-- (5) Calls chromaService.js -> stores chunks in ChromaDB
|-- (6) Queries ChromaDB -> retrieves top 10 relevant chunks
|-- (7) Calls groqService/geminiService -> dynamically builds prompt, calls AI
APIs
|-- (8) AI returns JSON array of questions
|-- (9) Calls historyModel.js -> saves JSON to SQLite, increments usage stats
|-- (10) Deletes PDF from disk & Deletes ChromaDB collection to free memory
v
[ Browser ] <- Returns JSON Response
|
|-- (11) User clicks Download -> POST /api/export
v
[ exportController.js ]
|
|-- (12) Saves JSON to temp file
|-- (13) Spawns python export_service.py
|-- (14) Python generates DOCX/PDF
v
[ Browser ] <- Streams binary file for download

```

SECTION 16 – Complete Execution Timeline

- **Second 0:** User clicks "Generate Bank". Frontend sends POST request.
 - **Second 1:** Backend receives request; Multer streams PDF to local disk.
 - **Second 2:** `pdfService.js` reads the file and extracts raw text.
 - **Second 3:** Text is chunked using sliding window overlap.
 - **Second 5:** Chunks are sent to Gemini `text-embedding-004`; vectors are returned.
 - **Second 8:** Vectors and metadata are stored in local ChromaDB.
 - **Second 9:** Backend performs Cosine Similarity search on ChromaDB.
 - **Second 10:** Top-K relevant chunks are retrieved.
 - **Second 11:** `geminiService.js` constructs the massive, rule-bound prompt injecting the chunks.
 - **Second 12:** Request dispatched to Groq API (or Gemini for 16-marks).
 - **Second 20:** AI model finishes streaming the generated JSON response.
 - **Second 22:** Controller parses JSON, applies regex fallback formatting (math sanitization).
 - **Second 24:** SQLite `question_banks` table is updated with the history; stats incremented.
 - **Second 25:** Temporary PDF and Chroma collection are deleted. JSON is returned to React.
 - **Second 27:** (User clicks Download) -> Backend spawns `export_service.py`.
 - **Second 29:** `reportlab` normalizes unicode and compiles the PDF layout.
 - **Second 30:** PDF is piped back as a binary stream to the browser. Download completes.
-

SECTION 17 – Interview Questions

Here are 10 highly technical interview questions (condensed from 50 for readability, focusing on the core architectural pillars) based on this project:

Q1: Why did you choose a hybrid Node.js and Python architecture instead of writing everything in Django or FastAPI? *Answer:* Node.js (Express) handles asynchronous I/O incredibly well, making it perfect for managing concurrent API requests, streaming file uploads via Multer, and making non-blocking network calls to the AI APIs. However, Python has superior, battle-tested libraries for document generation ([reportlab](#), [python-docx](#)). By using Node.js as the orchestrator and spawning Python purely as an isolated microservice script, we get the best of both ecosystems—high concurrency for networking and robust formatting for file generation.

Q2: Explain the "Sliding Window" chunking strategy and why it's critical for RAG (Retrieval-Augmented Generation). *Answer:* If we strictly split text every 1,000 characters, we risk slicing a critical mathematical formula or a sentence in half, destroying its semantic meaning. A sliding window introduces overlap (e.g., 200 characters). The end of chunk A becomes the beginning of chunk B. This ensures context boundaries are preserved, meaning the embedding model can accurately capture the full semantic meaning of the concept without truncation.

Q3: How does ChromaDB retrieve "relevant" information? Explain the math briefly. *Answer:* ChromaDB stores high-dimensional vectors representing text. When a query is made, it is also converted into a vector. ChromaDB calculates the Cosine Similarity between the query vector and all stored vectors. Cosine similarity measures the cosine of the angle between two vectors in multi-dimensional space. An angle of 0° (cosine 1) means they are pointing in the exact same direction (highly similar context). Chroma returns the "Top-K" chunks with the highest cosine similarity scores.

Q4: Your application uses Groq, but implements a Gemini fallback. Why? *Answer:* Groq uses specialized LPU hardware, making inference (generation) incredibly fast, which provides an excellent UX. However, Groq's free tier imposes a strict 12,000 Tokens Per Minute (TPM) limit. Generating a large question bank easily exceeds this. Therefore, a fallback mechanism is engineered: if the backend catches a 429 Rate Limit error from Groq, it gracefully swallows the exception and re-routes the prompt to the Gemini API, ensuring the user's request never fails due to infrastructure limits.

Q5: Why did you choose SQLite over PostgreSQL for this specific architecture? *Answer:* BloomAI currently functions as a local, single-tenant application. SQLite is a zero-configuration, serverless, file-based database ([database.sqlite](#)). It requires no background services, no port configurations, and no Docker containers to run, minimizing deployment friction. While PostgreSQL offers better concurrent write performance and row-level locking, those features are over-engineering for a local tool that primarily performs simple sequential reads and writes of history logs.

Q6: What issue did you face with PDF generation via ReportLab, and how was it solved? *Answer:* The AI models occasionally output mathematical expressions using Unicode subscripts or superscripts (e.g., x_1 as x_1). The default ReportLab fonts (like Helvetica) lack these specific Unicode glyphs, resulting in black squares (■) rendering on the PDF. This was solved by preprocessing the text through Python's `unicodedata.normalize('NFKD', text)`, which safely decomposes special Unicode characters into their standard ASCII equivalents (e.g., x_1 becomes $x1$) before PDF rendering.

Q7: How do you prevent the AI from generating duplicate questions in large sets? *Answer:* Instead of generating all questions simultaneously and filtering them post-generation, the architecture uses an iterative prompt-injection strategy. It generates the high-mark (16-mark) questions first. It then extracts the topics and exact texts of those generated questions and injects them into the system prompt for the subsequent (2-mark) generations under an **EXCLUSION RULES** directive, instructing the AI explicitly to avoid those topics. This prevents duplicates at the model-inference level.

Q8: Why must you extract text using Gemini Vision OCR instead of just pdf-parse for mathematical documents? *Answer:* **pdf-parse** reads the underlying text layer of a PDF. However, mathematical formulas, fractions, and complex matrices in PDFs are often rendered as vector graphics, embedded images, or custom non-standard fonts. **pdf-parse** will output corrupted garbage for these. Gemini Vision OCR analyzes the visual rendering of the page, accurately translating complex spatial math symbols back into coherent text.

Q9: How is the Custom User Instruction implemented safely without breaking the JSON output?

Answer: The custom instruction string is appended to the master prompt under a specific, walled-off section **USER CUSTOM INSTRUCTIONS (CRITICAL):**. Because the core system prompt strictly defines the JSON schema output format *after* this instruction block, the AI treats the custom instruction as a qualitative modifier (e.g., "focus on page 5") without allowing it to overwrite the hardcoded structural JSON requirements.

Q10: What is the purpose of multer in the Express backend? *Answer:* HTTP requests sending files from the browser use **multipart/form-data** encoding. Native Express cannot parse this directly. **multer** is a middleware that intercepts the incoming binary stream, buffers it, and writes the PDF file securely to the local server disk (**backend/src/uploads**), attaching the file metadata to the **req.file** object so the controller can pass the path to the text extraction services.